Interview Guide: Custom Heap Memory Allocator

Part 1: Code Explanation (Under the Hood)

1. The Metadata Struct

```
struct block_meta {
    size_t size;
    int is_free;
    struct block_meta *next;
};
```

Why we need it: When a user calls `my_free(ptr)`, they don't provide the size of the block. The OS needs a way to remember. We attach this hidden "header" directly before every block of memory to store its size, status, and a pointer to the next block (forming a Linked List).

2. Memory Alignment

```
#define ALIGNMENT 8
#define ALIGN(size) (((size) + (ALIGNMENT-1)) & ~(ALIGNMENT-1))
```

What it does: Rounds up requested bytes to the nearest multiple of 8.
Why it matters: Processors (like ARM chips in TI devices) read memory in chunks. If data starts on an odd address, the CPU takes multiple cycles to read it or throws a "hardware alignment fault." We align everything to 8 bytes to guarantee compatibility.

3. Requesting Memory (`sbrk`)

```
block = sbrk(0);
void *request = sbrk(size + META_SIZE);
```

What it does: The operating system gives every program a "data segment" (the heap). `sbrk(0)` tells us the current end address. `sbrk(size)` asks the OS kernel to physically stretch the heap, allocating raw memory.

4. The Pointer Math (Critical)

```
return (block + 1); // inside my_malloc
struct block_meta* block_ptr = (struct block_meta*)ptr - 1; // inside my_free
```

In `my_malloc`: We cannot return the metadata pointer, or the user will overwrite it. Adding `1` to a struct pointer moves the address forward by exactly `sizeof(struct block_meta)`, hiding the header and returning the safe data region.
In `my_free`: We cast the user's pointer to a `block_meta` pointer and subtract `1`. This steps *backward* in memory, landing perfectly on our hidden metadata.

Part 2: Top Interview Questions & Answers

1. What is the time complexity of your malloc and free functions?

"Since I am managing the heap as an explicit singly linked list, `my_malloc` uses a First-Fit algorithm. This means allocation is **O(N)** because it traverses the list to find a free block. My `my_free` function is also **O(N)** because I implemented a linear scan to coalesce adjacent blocks."

2. How would you optimize this for O(1) time complexity?

"A linear linked list is too slow for real-time systems. I would upgrade the architecture to use **Segregated Free Lists**. Instead of one list, I would create an array of lists bucketed by size (e.g., 16 bytes, 32 bytes). This allows O(1) lookup times for specific memory block sizes."

3. What is the difference between Internal and External Fragmentation?

- **Internal Fragmentation:** Memory wasted *inside* a block. Because I align to 8 bytes, asking for 13 bytes allocates 16. The 3 wasted bytes are internal fragmentation.
- **External Fragmentation:** Memory wasted *between* blocks. If the heap is chopped into tiny free pieces, a large allocation fails. Block coalescing in my `free()` function prevents this.

4. Why did you use sbrk() instead of mmap()?

"I used `sbrk()` because it naturally builds a single, contiguous heap, making linked-list traversal and coalescing cleaner for a foundational project. However, modern systems prefer `mmap()` because mapping isolated pages directly from the OS is safer."

5. Why is standard malloc dangerous in RTOS / Embedded Systems?

"Determinism and Fragmentation. Standard `malloc` searches a heap, so execution time varies randomly —unacceptable for microsecond-critical radar systems. Furthermore, long-running systems suffer from external fragmentation, eventually crashing the microcontroller. RTOSs use pre-allocated static memory pools instead."

> 💡 **Interview Strategy Pivot**
>
> When the interviewer asks about your background, leverage your CP rating but pivot immediately to systems. **Say this:**
>
> *"I have a very strong algorithmic background with over 1,500 problems solved, but I am pivoting hard into systems engineering. I built a distributed file system, and to understand the lowest levels of OS behavior, I engineered my own heap memory allocator in C to handle POSIX system calls and block coalescing."*